Part 2: Indexing and Evaluation

Group members: Júlia Hernan (u233681), Alan Le Roux (u233515), Alba Miró(u233727)

GitHub repository: https://github.com/juliahupf/RIAW.git

1. Introduction

This report describes the second phase of our search engine. Starting from the 28,080 documents pre processed in Part 1, we build an inverted index, answer conjunctive (AND) queries and rank the results with TF-IDF. Then we implement seven evaluation metrics, apply them to the two validation queries and to five test queries of our own, for which we create the relevance judgments ourselves. The last sections analyse what the metrics say about the system and which problems it has.

The search code is in myapp/search/algorithms.py, the metrics in myapp/search/evaluation.py, and all the experiments in project_progress/part_2/part2.ipynb. Every output shown in this report is a capture of a cell of that notebook. Section 5 explains how to reproduce them.

2. Indexing

2.1 Query pre-processing

A query must be processed exactly like the documents: if a user writes "running" and the index stores the stem run, there is no match. The function build_terms(text) therefore repeats the pipeline of Part 1: case folding, URL removal, punctuation replaced by spaces, splitting on whitespace, NLTK English stop-word removal, removal of single letters (digits are kept) and Porter stemming. For example, "men running shoes" becomes ['men', 'run', 'shoe']. The function is copied from the Part 1 notebook because code inside a notebook cannot be imported from a Python module.

2.2 Inverted index and conjunctive queries (Task 1)

create_inverted_index(docs, field) builds the index on all_terms, the field that Part 1 created by merging title, description, brand, category, sub_category and product_details. The statement suggests storing, for each term, the list of documents that contain it. We store that list as the keys of a dictionary, and the number of occurrences of the term in each document (term frequency) as the values, because TF-IDF needs it: {term: {pid: tf}}.

conjunctive_match(query_terms, index) implements the AND semantics required by the statement: a document is returned only if it contains every query term. It intersects the posting lists of the query terms, starting with the shortest one, because the result can never be larger than that list. If a query term is not in the vocabulary, no document can contain it and the result is empty.

```
Vocabulary size: 6768
Postings of 'kurta' (first 3): [('ETHFYB46QHB6PMYG', 7), ('KTAFTMYM2VYNZACZ', 5), ('ETHFSRVEN6JGSBYG', 4)]
Query terms: ['women', 'cotton', 'kurta']
Documents with each term: {'women': 13807, 'cotton': 19918, 'kurta': 1196}
Documents with all terms: 575
```

Output 1. Building the index and an example of a conjunctive query.

Output 1 shows that the vocabulary has 6,768 terms, and three postings of kurta: each pid comes with the number of times the term appears in that product. For "women cotton kurta", women appears in 13,807 documents, cotton in 19,918 and kurta in 1,196, but only 575 documents contain the three terms. The AND filter discards more than half of the documents that contain the rarest term, which shows how strict conjunctive queries are.

2.3 Test queries (Task 2)

We chose the query words by popularity. Output 2 lists the 15 terms with the highest document frequency (df), with their collection frequency (cf, total occurrences) and their idf. These terms are useless as query words: cloth appears in 27,722 of the 28,080 documents, so its idf is 0.020 and it cannot separate one product from another. Most of them come from the category and from the field names of product_details, which appear in almost every product. In other words, the most popular terms of the collection describe the structure of the data, not the products.

	df	cf	idf
cloth	27722	31084	0.019
accessori	27189	29204	0.047
color	26862	37498	0.064
fabric	26450	57370	0.086
pattern	25783	26595	0.123
care	22642	23746	0.311
wear	20971	31691	0.421
wash	20528	26831	0.452
cotton	19918	34079	0.495
suitabl	19309	19507	0.540
western	19118	19189	0.555
sleev	18901	51332	0.571
fit	18834	39734	0.576
type	18189	38731	0.626
ideal	17198	17770	0.707

Output 2. The 15 terms with the highest document frequency.

We therefore looked at popular words that describe a product. They range from very common terms, such as men and women, with an idf close to 1, to rarer and more discriminative ones, such as run, sneaker and shoe, with an idf above 5. Each query mixes both kinds, so that we can test whether the

ranking relies on the informative terms. Note that tshirt appears in only 2,648 documents although T-shirts are the most common product:

	df	cf	idf
cotton	19918	34079	0.495
shirt	16770	50038	0.744
neck	15190	57363	0.886
men	14250	28248	0.979
women	13807	28041	1.024
white	3937	6626	2.834
polo	3054	11489	3.201
winter	2868	3287	3.291
tshirt	2648	4009	3.407
hood	2161	3600	3.700
kurta	1196	4202	4.553
jacket	1110	3338	4.661
run	792	916	5.148
sneaker	780	803	5.170
shoe	526	1346	5.738

Output 3. Popularity of the product words used to build the test queries.

The five test queries (screenshot below) were chosen to cover different situations:

- **Query 3, "women cotton kurta":** a popular ethnic product combined with a gender and a fabric.
- **Query 4, "men running shoes":** footwear, a small part of the corpus
- **Query 5, "polo neck tshirt men":** a very popular garment, written as "tshirt".
- **Query 6, "winter hooded jacket men":** an attribute that is stored in product_details (Hooded).
- **Query 7, "white sneakers women":** a colour combined with a rare product type.

	query_id	query	terms	retrieved
0	1	women full sleeve sweatshirt cotton	[women, full, sleev, sweatshirt, cotton]	500
1	2	men slim jeans blue	[men, slim, jean, blue]	222
2	3	women cotton kurta	[women, cotton, kurta]	575
3	4	men running shoes	[men, run, shoe]	76
4	5	polo neck tshirt men	[polo, neck, tshirt, men]	203
5	6	winter hooded jacket men	[winter, hood, jacket, men]	292
6	7	white sneakers women	[white, sneaker, women]	76

Output 4. Validation (1, 2) and test (3 to 7) queries, their terms and the number of documents retrieved.

The number of retrieved documents depends on how rare the rarest term is and on how often the terms appear together. Queries 4 and 7 retrieve only 76 documents each, because shoe and sneaker are rare, while Query 3 retrieves 575, because kurtas are numerous and almost all of them are described with women and cotton.

2.4 Ranking with TF-IDF (Task 3)

We use the vector space model: the query and every document are vectors with one weight per term, and documents are ranked by the cosine of the angle between both vectors. The weights follow the TF IDF scheme with a logarithmic term frequency, in base 2 as seen in the theory sessions. The same formulas are applied to documents and queries.

We use TF-IDF to measure how important a term is in a document. Term Frequency (TF) gives more weight to frequently occurring terms, using logarithmic scaling to avoid overcounting. Inverse Document Frequency (IDF) gives more importance to rare terms and less to common ones. Their product gives the TF-IDF weight. Finally, cosine similarity compares query and document vectors to rank results, normalising for document length.
The formulas are in four functions:

- tf_weight(tf) returns 1 + log2(tf)
- create_tfidf_index(docs) builds the index and computes, once and before any query, the idf of every term) and the norm |d| of every document .
- score_tfidf(...) weights the query terms (their tf in the query is counted with Counter), computes |q| and the cosine of every document that passed the AND filter.
- search_in_corpus(query, ...) runs build_terms, the AND filter and score_tfidf, and sorts by decreasing score. Ties are broken by pid, so that every run returns the same ranking.

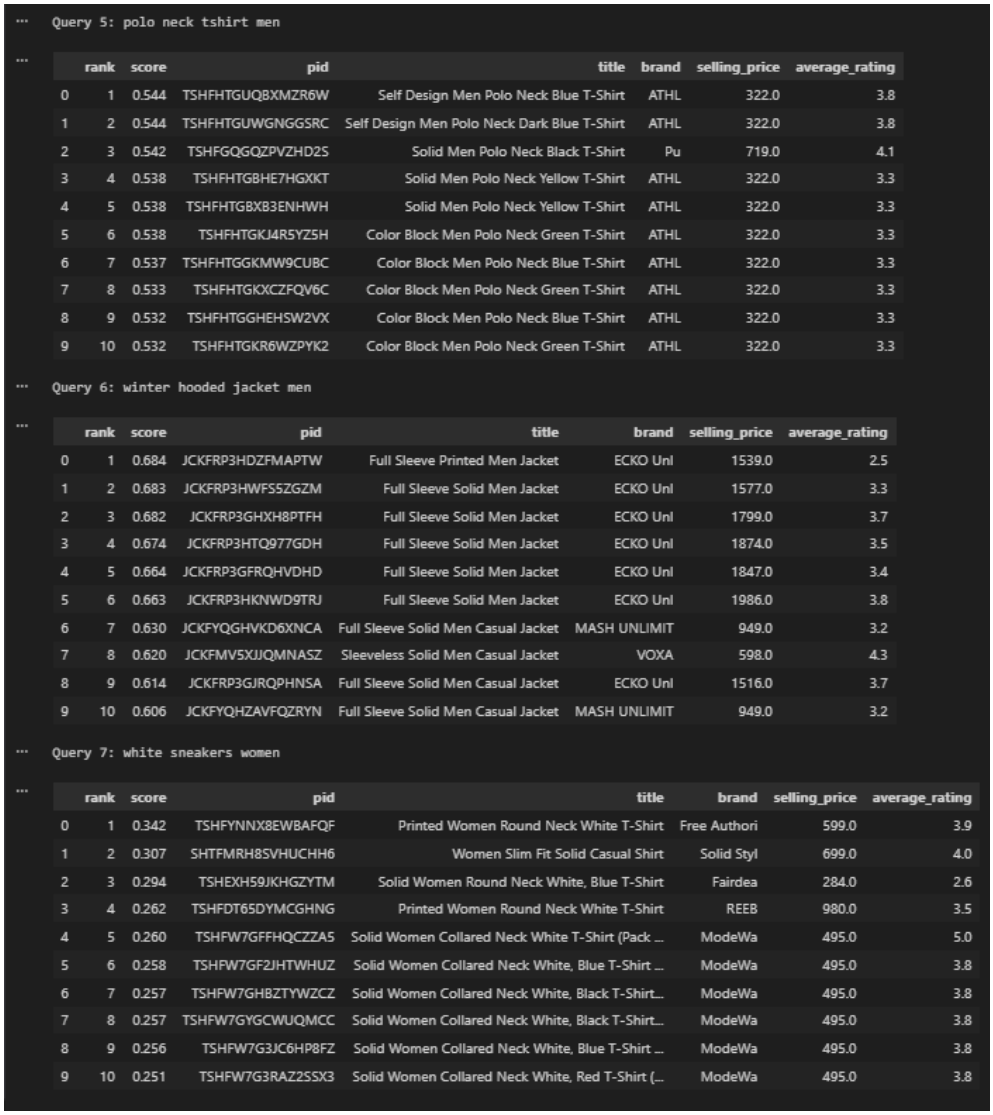
Output 5. First ten results of Queries 3 and 4.

Query 3: women cotton kurta

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.522	KTAFAKGUDJEB5DR7	Women Solid Cotton Blend Straight Kurta (Black)	Purple Sta	719.0	2.9
1	2	0.505	ETHFPDUMHA2BTSZH	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
2	3	0.505	ETHFPDUMHFZMYHCX	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
3	4	0.505	ETHFPDUMQZ9EZJEH	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
4	5	0.505	ETHFPDUMTE4P2QW9	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
5	6	0.505	ETHFPDUMZXGVUJFT	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
6	7	0.505	ETHFPDUMTGC3VF9M	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
7	8	0.504	ETHFPDUMFNYH3EUX	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
8	9	0.501	ETHFPDUMEWUYJGKR	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9
9	10	0.501	ETHFPDUMQAXJKDWZ	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	899.0	3.9

Query 4: men running shoes

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.599	SHOEP5GUQYYFKYSN	Running Shoes For Women (White)	Vector	990.0	4.1
1	2	0.598	SHOESS9ZEZ6TEQUJ	Running Shoes For Men (Multicolor)	Vector	1440.0	4.3
2	3	0.598	SHOESS9ZGMH7CURM	Running Shoes For Men (Multicolor)	Vector	1440.0	3.7
3	4	0.438	SHOEWQKVSGFGXPQF	Running Shoes For Men (Blue)	Vector	1520.0	3.3
4	5	0.429	SHOEKNRJGEYJCDQK	SPRINT Running Shoes For Men (Blue, Grey)	Vector	490.0	3.7
5	6	0.428	SHOEKNRJ34UCMAPA	SPRINT Running Shoes For Men (Black, Grey)	Vector	490.0	3.7
6	7	0.407	SHOFWFVWRSNK5AZJ	Running Shoes For Women (Multicolor)	Bone	474.0	3.9
7	8	0.403	SHOEYF5UHUUH5ADZ	Running Shoes For Women (Blue, Orange)	Vector	990.0	4.3
8	9	0.354	SHOFTZFPNCU4VWHK	Running Shoes For Men (Black)	Bone	379.0	4.1
9	10	0.349	SHOFTZFQP8W9G86F	Running Shoes For Women (Black)	Bone	341.0	4.1



Query 5: polo neck tshirt men

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.544	TSHFHTGUQBXMZR6W	Self Design Men Polo Neck Blue T-Shirt	ATHL	322.0	3.8
1	2	0.544	TSHFHTGUWGNGGSRC	Self Design Men Polo Neck Dark Blue T-Shirt	ATHL	322.0	3.8
2	3	0.542	TSHFGQGQZPVZHD2S	Solid Men Polo Neck Black T-Shirt	Pu	719.0	4.1
3	4	0.538	TSHFHTGBHE7HGXKT	Solid Men Polo Neck Yellow T-Shirt	ATHL	322.0	3.3
4	5	0.538	TSHFHTGBXB3ENHWH	Solid Men Polo Neck Yellow T-Shirt	ATHL	322.0	3.3
5	6	0.538	TSHFHTGKJ4R5YZ5H	Color Block Men Polo Neck Green T-Shirt	ATHL	322.0	3.3
6	7	0.537	TSHFHTGGKMW9CUBC	Color Block Men Polo Neck Blue T-Shirt	ATHL	322.0	3.3
7	8	0.533	TSHFHTGKXCZFQV6C	Color Block Men Polo Neck Green T-Shirt	ATHL	322.0	3.3
8	9	0.532	TSHFHTGGHEHSW2VX	Color Block Men Polo Neck Blue T-Shirt	ATHL	322.0	3.3
9	10	0.532	TSHFHTGKR6WZPYK2	Color Block Men Polo Neck Green T-Shirt	ATHL	322.0	3.3

Query 6: winter hooded jacket men

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.684	JCKFRP3HDZFMAPTW	Full Sleeve Printed Men Jacket	ECKO Unl	1539.0	2.5
1	2	0.683	JCKFRP3HWFS5ZGZM	Full Sleeve Solid Men Jacket	ECKO Unl	1577.0	3.3
2	3	0.682	JCKFRP3GHXH8PTFH	Full Sleeve Solid Men Jacket	ECKO Unl	1799.0	3.7
3	4	0.674	JCKFRP3HTQ977GDH	Full Sleeve Solid Men Jacket	ECKO Unl	1874.0	3.5
4	5	0.664	JCKFRP3GFRQHVDHD	Full Sleeve Solid Men Jacket	ECKO Unl	1847.0	3.4
5	6	0.663	JCKFRP3HKNWD9TRJ	Full Sleeve Solid Men Jacket	ECKO Unl	1986.0	3.8
6	7	0.630	JCKFYQGHVKD6XNCA	Full Sleeve Solid Men Casual Jacket	MASH UNLIMIT	949.0	3.2
7	8	0.620	JCKFMV5XJJQMNASZ	Sleeveless Solid Men Casual Jacket	VOXA	598.0	4.3
8	9	0.614	JCKFRP3GJRQPHNSA	Full Sleeve Solid Men Casual Jacket	ECKO Unl	1516.0	3.7
9	10	0.606	JCKFYQHZAVFQZRYN	Full Sleeve Solid Men Casual Jacket	MASH UNLIMIT	949.0	3.2

Query 7: white sneakers women

	rank	score	pid	title	brand	selling_price	average_rating
0	1	0.342	TSHFYNNX8EWBAFQF	Printed Women Round Neck White T-Shirt	Free Authori	599.0	3.9
1	2	0.307	SHTFMRH8SVHUCHH6	Women Slim Fit Solid Casual Shirt	Solid Styl	699.0	4.0
2	3	0.294	TSHEXH59JKHGZYTM	Solid Women Round Neck White, Blue T-Shirt	Fairdea	284.0	2.6
3	4	0.262	TSHFDT65DYMCGHNG	Printed Women Round Neck White T-Shirt	REEB	980.0	3.5
4	5	0.260	TSHFW7GFFHQCZZA5	Solid Women Collared Neck White T-Shirt (Pack ...	ModeWa	495.0	5.0
5	6	0.258	TSHFW7GF2JHTWHUZ	Solid Women Collared Neck White, Blue T-Shirt ...	ModeWa	495.0	3.8
6	7	0.257	TSHFW7GHBZTYWZCZ	Solid Women Collared Neck White, Black T-Shirt...	ModeWa	495.0	3.8
7	8	0.257	TSHFW7GYGCWUQMCC	Solid Women Collared Neck White, Black T-Shirt...	ModeWa	495.0	3.8
8	9	0.256	TSHFW7G3JC6HP8FZ	Solid Women Collared Neck White, Blue T-Shirt ...	ModeWa	495.0	3.8
9	10	0.251	TSHFW7G3RAZ2SSX3	Solid Women Collared Neck White, Red T-Shirt (...	ModeWa	495.0	3.8

Output 6. First ten results of Queries 5, 6 and 7.

Outputs show the ranking produced for each test query. In **Query 3** the first result is a single kurta (score 0.522), followed by nine colour variants of the same "Women Kurta and Pyjama Set Pure Cotton" with almost identical scores (0.501 to 0.505): their texts are the same except for the colour, so TF-IDF cannot separate them. In **Query 4** the first result is a running shoe for women (0.599), just above two men's shoes (0.598), and three more women's shoes appear at ranks 7, 8 and 10: the query asks for men, but the ranking barely distinguishes gender. In **Query 5** all ten results are men's polo neck T-shirts, seven of them from the same brand (ATHL). In **Query 6** the scores are the highest of all queries (0.606 to 0.684), yet the first nine jackets have no hood; only the tenth does. In **Query 7** the scores are the lowest (0.251 to 0.342) and the ten results are white T-shirts and a shirt, not sneakers.

3. Evaluation

3.1 Evaluation metrics (Task 1)

All metrics are in myapp/search/evaluation.py. Each function receives the true labels y_true (1 relevant, 0 not relevant) and the scores y_score of the engine. The helper rank_labels sorts the labels by decreasing score with a stable sort. In the formulas, rel_i is the label at rank i, K the cut-off and R the total number of relevant documents.

We evaluate the search system using several ranking metrics: Precision@K measures how many of the top K results are relevant, while Recall@K measures how many relevant documents are retrieved. F1@K combines both metrics. Average Precision@K (AP) evaluates how well relevant documents are ranked, and MAP@K averages AP across all queries. MRR measures the position of the first relevant result, while NDCG@K evaluates ranking quality by giving more importance to relevant documents appearing earlier in the results.

```
y_true = np.array([0, 1, 0, 1, 1])
y_score = np.array([0.9, 0.5, 0.6, 0.7, 0.2])   # ranked labels: 0, 1, 0, 1, 1

assert precision_at_k(y_true, y_score, 2) == 1 / 2
assert recall_at_k(y_true, y_score, 2) == 1 / 3
assert np.isclose(f1_at_k(y_true, y_score, 2), 2 * (1 / 2) * (1 / 3) / (1 / 2 + 1 / 3))
assert np.isclose(avg_precision_at_k(y_true, y_score, 5), (1 / 2 + 2 / 4 + 3 / 5) / 3)
assert rr_at_k(y_true, y_score, 5) == 1 / 2
assert rr_at_k(y_true, y_score, 1) == 0
assert ndcg_at_k(y_true, y_true, 5) == 1
assert recall_at_k(y_true, y_score, 5, total_relevant=6) == 1 / 2   # relevant documents not retrieved count
print("Metric checks passed")
```

```
Metric checks passed
```

Output 7. Checks of the metrics on cases computed by hand.

How the rankings are evaluated. The function evaluate_query computes all metrics of one query for K = 5, 10 and 20, and evaluate adds MAP and MRR over a set of queries. For the validation queries only the 40 documents of validation_labels.csv are judged, so only those documents enter the ranking, in the order given by our engine. For our test queries every document of the corpus has a label , so the whole result list is evaluated. In both cases R counts the relevant documents that the AND filter did not return.

3.2 Results on the validation queries (Task 2)

query_id	1	2
P@5	1.000	0.800
R@5	0.385	0.400
F1@5	0.556	0.533
AP@5	1.000	1.000
RR@5	1.000	1.000
NDCG@5	1.000	0.869
P@10	1.000	0.600
R@10	0.769	0.600
F1@10	0.870	0.600
AP@10	1.000	0.948
RR@10	1.000	1.000
NDCG@10	1.000	0.716
P@20	0.650	0.300
R@20	1.000	0.600
F1@20	0.788	0.400
AP@20	0.980	0.948
RR@20	1.000	1.000
NDCG@20	0.995	0.716

k	MAP	MRR
5	1.000	1.000
10	0.974	1.000
20	0.964	1.000

Output 8. Metrics of the validation queries.

To compute recall correctly we need every relevant document of the corpus, also those that the engine misses, and judging 28.080 documents by hand for five queries is not feasible. We therefore wrote down each information need as an explicit rule and applied it to the whole collection.

A document is relevant if:

- **Query 3:** the title contains "women" and "kurta", and the Fabric detail contains cotton (a kurta alone or in a set).
- **Query 4:** the title contains "running shoes" and "for men".
- **Query 5:** the title describes a men's polo neck T-shirt, however "T-shirt" is written in the query.
- **Query 6:** it is a men's jacket of the sub_category Winter Wear and its Hooded detail is Yes.
- **Query 7:** the title contains "sneakers for women" and the Color detail contains white.

The rules use the title and the structured product_details, which describe the product itself. The description is not used, because it often mentions other products ("pair it with sneakers"). We then read the top 20 results of each query to check that the rule agreed with our own judgment. The labels are saved in test_labels.csv, with the same format as validation_labels.csv (title, pid, query_id, labels).

```python
def details(doc):
    flat = {}
    for item in doc["product_details"] or []:
        flat.update(item)
    return flat

def has(pattern, text):
    return re.search(pattern, text or "", re.IGNORECASE) is not None

def clean_title(doc):
    return " ".join(doc["title"].split())

RELEVANCE_RULES = {
    # a women's kurta (alone or in a set) whose fabric is cotton
    3: lambda d: has(r"\bwomen\b", d["title"]) and has(r"\bkurta\b", d["title"]) and has("cotton", details(d).get("Fabric")),
    # running shoes made for men, not for women
    4: lambda d: has(r"running shoes", d["title"]) and has(r"\bfor men\b", d["title"]),
    # a men's T-shirt with polo neck, however the word "T-shirt" is written
    5: lambda d: has(r"\bmen\b.*polo neck.*t-shirt", clean_title(d)),
    # a men's jacket for winter that really has a hood
    6: lambda d: has(r"\bmen\b", d["title"]) and has(r"\bjacket\b", d["title"]) and d["sub_category"] == "Winter Wear"
                 and details(d).get("Hooded") == "Yes",
    # women's sneakers in white
    7: lambda d: has(r"sneakers for women", clean_title(d)) and has("white", details(d).get("Color")),
}

judged_rows = []
for q in TEST_QUERIES:
    relevant = {d["pid"] for d in docs if RELEVANCE_RULES[q](d)}
    top_20 = {pid for pid, _ in results[q][:20]}
    judged_rows += [{"title": clean_title(doc_store[pid]), "pid": pid, "query_id": q, "labels": int(pid in relevant)}
                    for pid in sorted(relevant | top_20)]

test_labels = pd.DataFrame(judged_rows)
test_labels.to_csv("test_labels.csv", index=False)   # same format as validation_labels.csv

test_labels_by_query = {}
for q in TEST_QUERIES:
    relevant = set(test_labels[(test_labels["query_id"] == q) & (test_labels["labels"] == 1)]["pid"])
    test_labels_by_query[q] = {pid: int(pid in relevant) for pid, _ in results[q]} | {pid: 1 for pid in relevant}

pd.DataFrame([{"query_id": q, "query": TEST_QUERIES[q], "retrieved": len(results[q]),
               "relevant in corpus": sum(labels.values()),
               "relevant retrieved": sum(labels[pid] for pid, _ in results[q]),
               "relevant in top 20": sum(labels[pid] for pid, _ in results[q][:20])}
              for q, labels in test_labels_by_query.items()])
```

	query_id	query	retrieved	relevant in corpus	relevant retrieved	relevant in top 20
0	3	women cotton kurta	575	322	322	20
1	4	men running shoes	76	23	23	10
2	5	polo neck tshirt men	203	1326	186	20
3	6	winter hooded jacket men	292	49	49	4
4	7	white sneakers women	76	3	3	0

Output 9. Relevance rules and number of relevant documents per test query.

Output 9 already shows two very different problems. For Queries 3, 4, 6 and 7, "relevant retrieved" equals "relevant in corpus": the AND filter returns all the relevant documents, so any error is a ranking error. Their share in the result list, however, varies a lot: 322 of 575 (56%) for Query 3, 23 of 76 (30%) for Query 4, 49 of 292 (17%) for Query 6 and only 3 of 76 (4%) for Query 7. When we inspected the 253 non-relevant results of Query 3, 242 were dhotis and ethnic dresses, which contain kurta only because their sub_category is "Kurtas, Ethnic Sets and Bottoms". Query 5 is the opposite case: almost everything it retrieves is relevant (186 of 203, 92%), but 1,140 of its 1,326 relevant T-shirts are never retrieved, a recall of only 14% over the whole list. The number of relevant documents also ranges from 3 to 1,326, which matters when reading the recall-based metrics below.

3.3 Results on the test queries

query_id	3	4	5	6	7
P@5	1.000	0.800	1.000	0.000	0.000
R@5	0.016	0.174	0.004	0.000	0.000
F1@5	0.031	0.286	0.008	0.000	0.000
AP@5	1.000	0.679	1.000	0.000	0.000
RR@5	1.000	0.500	1.000	0.000	0.000
NDCG@5	1.000	0.661	1.000	0.000	0.000
P@10	1.000	0.600	1.000	0.100	0.000
R@10	0.031	0.261	0.008	0.020	0.000
F1@10	0.060	0.364	0.015	0.034	0.000
AP@10	1.000	0.703	1.000	0.100	0.000
RR@10	1.000	0.500	1.000	0.100	0.000
NDCG@10	1.000	0.573	1.000	0.064	0.000
P@20	1.000	0.500	1.000	0.200	0.000
R@20	0.062	0.435	0.015	0.082	0.000
F1@20	0.117	0.465	0.030	0.116	0.000
AP@20	1.000	0.642	1.000	0.167	0.000
RR@20	1.000	0.500	1.000	0.100	0.000
NDCG@20	1.000	0.512	1.000	0.147	0.000

k	MAP	MRR
5	0.536	0.500
10	0.561	0.520
20	0.562	0.520

Output 10. Metrics of the five test queries, and MAP and MRR over the five queries.

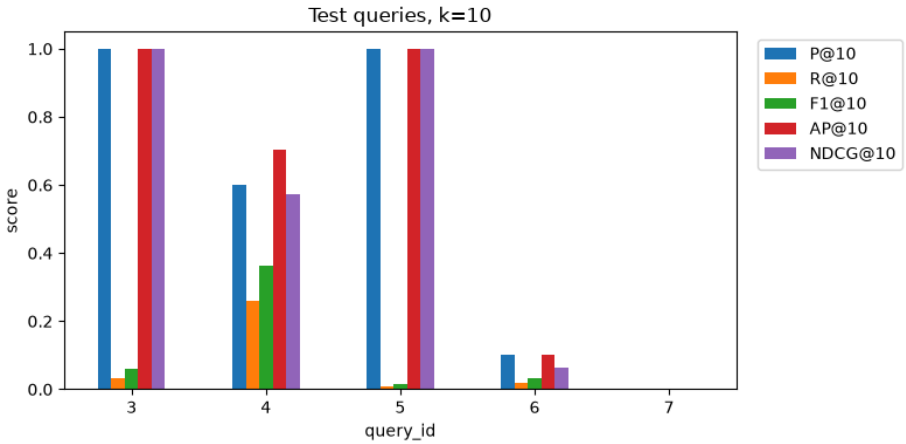


Figure 1. P, R, F1, AP and NDCG at K = 10 for each test query.

Figure 1 shows three very different behaviours.

Queries 3 and 5 have P@10, AP@10 and NDCG@10 equal to 1, meaning that the first ten results are all relevant. Their R@10 (0.031 and 0.008) and F1@10 (0.060 and 0.015) are almost zero, but this is not a failure of the ranking: with 322 and 1,326 relevant documents, ten results can find at most 10/322 = 0.031 and 10/1326 = 0.008 of them, which are exactly the values obtained.

Query 4 is the only one where all five bars are of medium height. P@10 = 0.600 means that 6 of the first 10 results are relevant. AP@10 (0.703) is higher than P@10 because the relevant documents are concentrated at ranks 2 to 6 (Output 5). NDCG@10 (0.573) is lower than AP@10 as NDCG compares with an ideal ranking of ten relevant documents and is penalised by the non-relevant documents at ranks 1, 7, 8 and 10. RR@10 = 0.500 because the first relevant document is at rank 2. R@10 = 0.261 is the

highest recall at K = 10, since there are only 23 relevant documents.

Query 6 has values close to zero (P@10 = 0.100, AP@10 = 0.100, NDCG@10 = 0.064, RR@10 = 0.100): only one of the first ten results is relevant and it is at rank 10.

Query 7 has no bars at all: no relevant document is in the top 10.

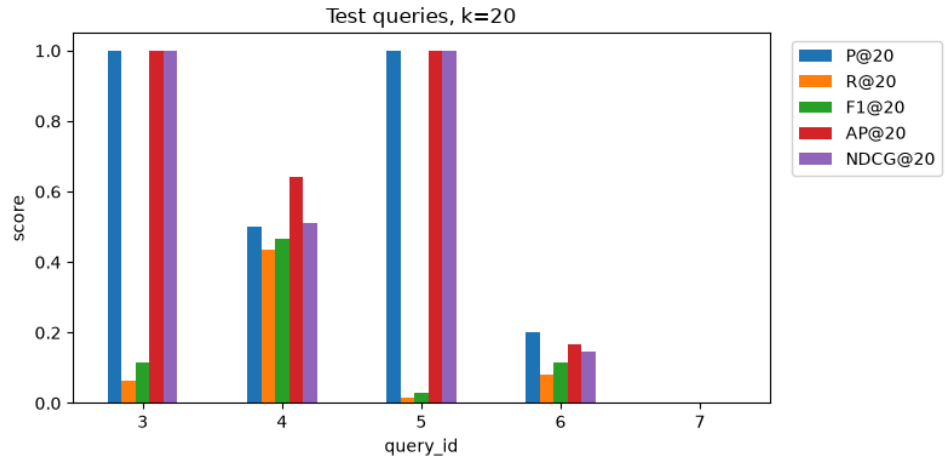


Figure 2. The same metrics at K = 20.

Figure 2 shows how the metrics change when the user looks at twice as many results. For Queries 3 and 5 precision stays at 1 for 20 results, and recall doubles (0.062 and 0.015). For Query 4 precision falls from 0.6 to 0.5 (only 4 of results 11 to 20 are relevant), while recall rises from 0.261 to 0.435, so F1 increases from 0.364 to 0.465, the best F1 of the five queries. AP@20 (0.642) and NDCG@20 (0.512) decrease because the new relevant documents arrive late. Query 6 improves slightly (P@20 = 0.2, AP@20 = 0.167) because three more relevant jackets appear, at ranks 13, 14 and 20. Query 7 remains at 0: its first relevant document is at rank 22, just outside the cut-off (Output 11). The difference between RR and AP is also visible: RR is 0.500 for Query 4 because only rank 1 is wrong, and 0.1 for Query 6 because the first relevant document is the tenth, while AP also rewards or penalises every later position.

3.5 Comments on the evaluation metrics

The metrics answer different questions about the same ranking:

- **P@K** tells how useful the first page is, but not in which order the results come or how many relevant documents are left outside it. It divides by K even when fewer than K documents are retrieved.
- **R@K** tells how much of the relevant material has been found, but its maximum is K/R, so it is low whenever a query has many relevant documents.
- **F1@K** is only high when precision and recall are both high, so it rewards balanced rankings
- **AP@K** measures whether the relevant documents of one query are at the top. With the practice definition it cannot see relevant documents that were never retrieved.
- **MAP** summarises the system in one number, at the cost of hiding the differences between queries.
- **RR and MRR** only look at the first relevant document, which suits a user who needs one good answer.
- **NDCG@K** compares with the ideal ranking and penalises errors at the top more than errors at

the bottom. Its ideal ranking includes all relevant documents, so it does notice the documents that the system misses.

Validation queries. Query 1 obtains P@10 = 1.000 and R@20 = 1.000: the first ten of the judged documents are relevant and the 13 relevant documents are almost entirely ahead of the 7 non-relevant ones (AP@20 = 0.980). This looks good, but it only means that the judged documents are well ordered among themselves. In the full ranking of our engine they appear between ranks 149 and 500 of 500 (Output 11), so the first page shows documents that nobody judged.

Query 2 shows why several metrics are needed. AP@10 = 0.948 and RR = 1.000 suggest an almost perfect system, while R@20 = 0.600 and NDCG@20 = 0.716 show that 4 of the 10 relevant documents were never retrieved. Only 8 of the 20 judged documents pass the AND filter, which also explains why P@20 drops to 0.300. AP cannot see the missing documents, while NDCG can, because its ideal ranking contains all of them. Three of the four missing documents are titled "Women" (for example "Slim Women Blue Jeans") and lack the term men; the fourth lacks slim.

Test queries. MAP@10 = 0.561 and MRR@10 = 0.520 are averages of very different behaviours: two queries with perfect precision (3 and 5), one medium query (4) and two failures (6 and 7). MAP also hides the recall problem of Query 5, since its AP is 1.000. Comparing this with the validation set, which has MAP@10 = 0.974, would be misleading: the validation metrics only rank 40 hand-picked documents, while the test metrics rank the full result lists.

3.6 Problems of the search system and proposed solutions

The numbers used in this section come from the last code cell of the notebook (Output 11).

```
Q5 relevant not retrieved: 1140 of 1326
Q2 judged documents retrieved: 8 of 20
t-shirt: ['shirt'] | tshirt: ['tshirt']
hooded: ['hood'] | hoodie: ['hoodi']
Q1 judged documents between ranks 149 and 500 of 500
Q7 ranks of the relevant documents: [22, 52, 58]
Q6 Hooded values (all results): Counter({'No': 181, 'Yes': 68, None: 43})
Q6 Hooded values (top 20): Counter({'No': 16, 'Yes': 4})
Q4 first result: Running Shoes For Women (White) ['footwear', 'men', 'footwear']
Q4 shoes for women in the top 20: 9
Q3 different titles in the top 20: 2
Out of stock in the top 20 of the test queries: 8
```

Output 11. Measurements used in the analysis of the search system.

1. Retrieval accuracy. The AND filter is very strict: if a document does not contain one of the query words, it is lost even if it is relevant. In validation Query 2 only 8 of the 20 judged documents are retrieved, and in Query 5 we lose 1,140 of the 1,326 relevant T-shirts. *Solution:* use AND first, and when there are few results, switch to an OR ranking where documents that contain more query terms get a higher score.

2. Ranking quality. A word counts the same in the title as in the description. In Query 7 the three white sneakers are at ranks 22, 52 and 58, behind T-shirts whose description only says they go well with

sneakers. *Solution:* give more weight to the title than to the description, for example with BM25F.

3. Handling of different field types. product_details is indexed as plain text, so “Hooded: No” adds the term hood to jackets without a hood. In Query 6, 181 of the retrieved jackets have no hood and only 68 have one, and 16 of the top 20 have no hood. *Solution:* keep attributes such as Hooded, Color or Fabric as fields and use them as filters.

4. Query formulation. The same word can be written in different ways. “t-shirt” becomes shirt and “tshirt” stays as tshirt, so a user gets different results depending on how they write it. The same happens with “hooded” (hood) and “hoodie” (hoodi). *Solution:* normalise these spellings before indexing and add a small dictionary of synonyms for fashion words.

5. Indexing strategy. We index everything in one field (all_terms), including the category. The sub_category “Men’s Footwear” also contains women’s shoes, so they get the term men: the first result of Query 4 is a women’s shoe, and 9 of the top 20 are for women. *Solution:* index the fields separately, as we proposed in Part 1, give the category a low weight and filter by the Ideal For detail.

6. Other: repeated products and stock. In Query 3 the top 20 contain only 2 different titles, because the same kurta appears in many colours. Also, 8 of the top-20 results of our queries are out of stock. *Solution:* group the colour variants of a product and move out-of-stock products down the ranking.

4. Conclusions and implications for the next phases

- **The AND filter and the ranking fail in different queries.** For four of the five test queries the filter keeps all relevant documents and the problem is the order. For Query 5 the order is perfect but 86% of the relevant documents are never retrieved. A single metric such as MAP does not show this; precision, recall and the full result list must be read together.
- **Most errors come from how fields are turned into text.** Category names (men, kurta) and attribute names (hood) produce matches that do not describe the product. Field-weighted ranking and structured attribute filters are the main improvements for Part 3.
- **Pre-processing must handle domain spellings**, such as "t-shirt" and "tshirt", before any ranking model can help.
- **Non-textual signals** (stock, rating) and result diversification should enter the final score.

5. Reproducibility

The README in project_progress/part_2 gives step-by-step instructions. The notebook part2.ipynb reads processed_docs.json from Part 1, builds the index, runs every query and writes test_labels.csv and the figures in evaluation_output/. Rankings are deterministic, because ties are broken by pid.

6. AI usage statement

We have used AI as a supporting tool to help us identify and fix syntax errors in the code, proofread and improve written texts, and assist us in preparing and structuring the README file.